

RWorksheet_prado#4b.Rmd

Prado, James Harry U.

2025-10-20

```
# Q1: Create a 5x5 matrix using a for loop

# 1. Initialize the 5x5 zero matrix
matrix_size <- 5
zero_matrix <- matrix(0, nrow = matrix_size, ncol = matrix_size)

# 2. Create the vectorA and index (0-based for the math)
vectorA <- c(1, 2, 3, 4, 5)

# 3. Use nested for loops to fill the matrix based on the difference formula
# The resulting value at A[i, j] should be |(i-1) - (j-1)|
# where i and j are the R indices (1-based)
for (i in 1:matrix_size) {
  for (j in 1:matrix_size) {
    # Calculate the absolute difference between the 0-based row and column indices.
    # (i - 1) gives 0, 1, 2, 3, 4
    # (j - 1) gives 0, 1, 2, 3, 4
    zero_matrix[i, j] <- abs((i - 1) - (j - 1))
  }
}

print("Resulting 5x5 Matrix:")
```

```
## [1] "Resulting 5x5 Matrix:"
```

```
zero_matrix
```

```
##      [,1] [,2] [,3] [,4] [,5]
## [1,]    0    1    2    3    4
## [2,]    1    0    1    2    3
## [3,]    2    1    0    1    2
## [4,]    3    2    1    0    1
## [5,]    4    3    2    1    0
```

```
# Q2: Print the string "*" pattern (Right-aligned triangle)
n <- 5 # Define the height of the pattern
```

```
for (i in 1:n) {
  # Print spaces: (n - i) spaces are needed
  spaces <- paste(rep(" ", n - i), collapse = "")

  # Print stars: i stars are needed
  stars <- paste(rep("*", i), collapse = "")
```

```

# Combine and print
cat(spaces, stars, "\n", sep = "")
}

##      *
##     **
##    ***
##   ****
##  *****

# Q3a: Create the data frame for product variants
Variation <- c("Black", "Black Plus", "Black Show", "Black Ultimate",
              "White", "White Plus", "White Show", "White Ultimate")
Total <- c(261, 270, 265, 275, 258, 272, 260, 277)

df_variants <- data.frame(Variation, Total)
print("Product Variants Data Frame:")

## [1] "Product Variants Data Frame:"
df_variants

##      Variation Total
## 1      Black    261
## 2  Black Plus    270
## 3  Black Show    265
## 4 Black Ultimate    275
## 5      White    258
## 6  White Plus    272
## 7  White Show    260
## 8 White Ultimate    277

# Q3b: Get the total number of each variation and save it
# This step requires the dplyr package for consistent R code practice,
# although the data is already aggregated. We will use the existing df_variants.

# Install and load dplyr (if not already installed)
# install.packages("dplyr")
library(dplyr)

##
## Attaching package: 'dplyr'

## The following objects are masked from 'package:stats':
##
##   filter, lag

## The following objects are masked from 'package:base':
##
##   intersect, setdiff, setequal, union

# The data is already in the required format (Variation and Total counts),
# so we simply assign it to the new object `variations` and save it.
variations <- df_variants

print("Variations data object:")

## [1] "Variations data object:"

```

```

print(variations)

##          Variation Total
## 1          Black    261
## 2      Black Plus    270
## 3      Black Show    265
## 4 Black Ultimate    275
## 5          White    258
## 6      White Plus    272
## 7      White Show    260
## 8 White Ultimate    277

# Save the object as variations.RData
save(variations, file = "variations.RData")
cat("\nObject 'variations' saved to 'variations.RData'.")

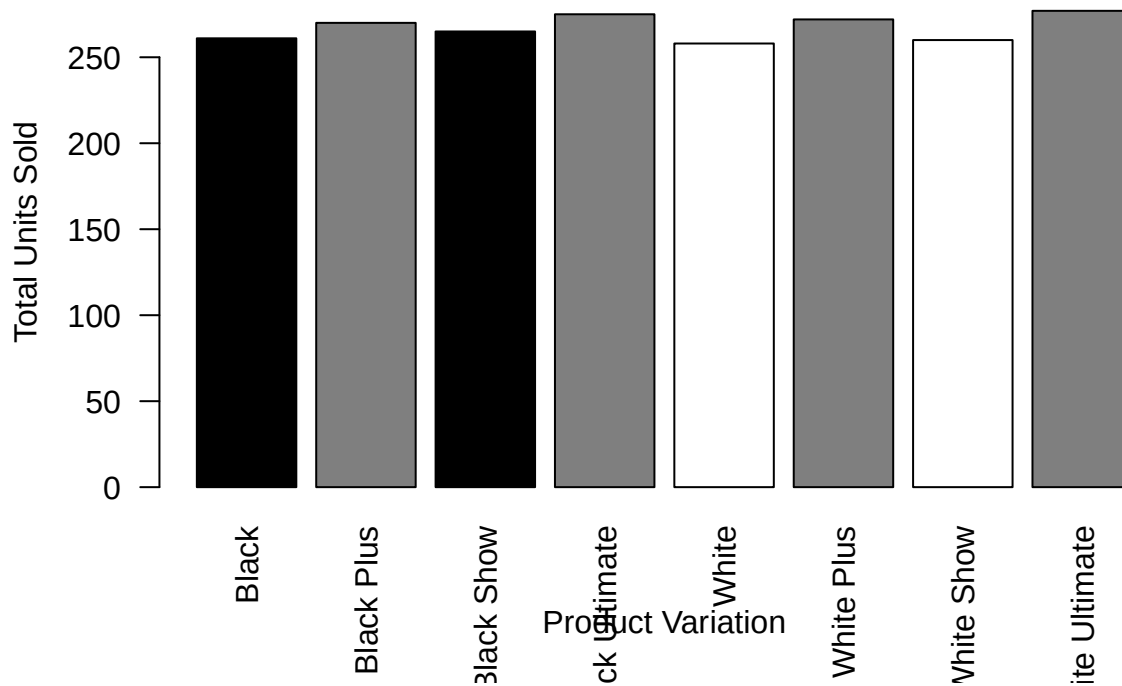
##
## Object 'variations' saved to 'variations.RData'.

# Q3c: Create a barplot for all variations

# Set colors: alternating black and white scheme, or distinct colors for all 8
bar_colors <- c("black", "gray50", "black", "gray50", "white", "gray50", "white", "gray50")
# Use the Total column for heights and Variation column for names (labels)
barplot(variations$Total,
        names.arg = variations$Variation,
        main = "Total Sales per Product Variation",
        ylab = "Total Units Sold",
        xlab = "Product Variation",
        col = bar_colors,
        border = "black",
        ylim = c(0, max(variations$Total) + 20),
        las = 2 # Rotate labels 90 degrees for better readability
)

```

Total Sales per Product Variation



Q3d: Create a barplot for the black and white variations, side-by-side

Separate the data into Black and White groups

```
black_variants <- variations[1:4, ]
```

```
white_variants <- variations[5:8, ]
```

Combine the 'Total' columns into a matrix for side-by-side plotting

The matrix must have rows = (Black, White) and columns = (Plus, Show, Ultimate)

To plot them side-by-side by group, we need a specific matrix structure:

Since the chart asks for Black and White variants, we can plot the Black vs White totals for each subgroup

Create a matrix where rows are the subgroups and columns are the main color categories

```
subgroup_names <- c("Base", "Plus", "Show", "Ultimate")
```

Data organization for grouped bar chart:

```
sales_matrix <- matrix(
  c(black_variants$Total, white_variants$Total), # Fill by column (default)
  nrow = 4, # 4 rows for Base, Plus, Show, Ultimate
  ncol = 2, # 2 columns for Black, White
  byrow = FALSE
)
```

```
rownames(sales_matrix) <- subgroup_names
```

```
colnames(sales_matrix) <- c("Black Variants", "White Variants")
```

Generate the grouped barplot

```
barplot(sales_matrix,
  main = "Sales Comparison: Black vs. White Variants",
```

```

ylab = "Total Units Sold",
col = c("gray20", "gray40", "gray60", "gray80"), # Different shades for subgroups
legend.text = rownames(sales_matrix),
beside = TRUE, # Key for side-by-side bars
ylim = c(0, max(sales_matrix) + 20)
)

```

Sales Comparison: Black vs. White Variants

